

4. Perform the following tasks using git-bash

- Initialize a Git repository for a data science project
- Push it to GitHub to enable version control
- Create a new branch named GIT-EXAM-1 and create new file named as program2 and write a program to check if given string is palindrome
- Create a python file name as program1.py and write a python program to print squares of numbers from 1 to 10
- Commit the code and push the changes and check the changes on remote repository (GitHub)
- Create a directory named as IGNORE and add test1.txt, test2.txt files into it
- Create .gitignore file and add the IGNORE directory into it to ignore the changes made to it
- Commit and push the code and demonstrate that IGNORE directory and its files are not getting pushed on remote

a. Open **Git Bash** and run:

```
mkdir data-science-project
```

```
cd data-science-project
```

```
git init
```

✅ b. Push it to GitHub

- Go to GitHub and create a new repository (e.g., data-science-project)
- Connect local repo to GitHub:

```
git remote add origin https://github.com/your-username/data-science-project.git
```

OR

```
git clone https://github.com/your-username/data-science-project.git
```

```
cd data-science-project
```

Open bash in cloned repo

```
echo "# Data Science Project" > README.md
```

```
git add .
```

```
git commit -m "Initial commit"
```

```
git branch -M main
```

```
git push -u origin main
```

✅ Step 3: Create Branch GIT-EXAM-1

```
git checkout -b GIT-EXAM-1
```

✅ Step 4: Create program2.py (Palindrome)

```
touch program2.py
```

```
code .
```

```
# Palindrome check
string = input("Enter string: ")
if string == string[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

✅ Step 5: Create program1.py (Squares 1–10)

```
touch program1.py
code .
```

```
# Squares from 1 to 10
for i in range(1, 11):
    print(i*i)
```

✅ Step 6: Commit & Push (GIT-EXAM-1)

```
git add .
git commit -m "Added palindrome and squares programs"
git push origin GIT-EXAM-1
```

✅ Step 7: Create IGNORE Directory & Files

```
mkdir IGNORE
```

open IGNORE folder

```
cd IGNORE
touch test1.txt test2.txt
cd ..
```

Step 8: Create .gitignore File

```
touch .gitignore
code .
```

👉 Add this line: IGNORE/

Step 9: Commit .gitignore

```
git add .gitignore
git commit -m "Added gitignore to ignore IGNORE folder"
git push origin GIT-EXAM-1
```

```
git status
```

👉 Output should show: nothing to commit, working tree clean

👉 Now check GitHub:

✓ program1.py → present

✓ program2.py → present

✓ .gitignore → present

✗ IGNORE/ folder → NOT present

“The IGNORE directory and its files are not pushed to GitHub because they are added in the .gitignore file, hence Git ignores them.”

IMPORTANT (Very Important for Exam) (OPTIONAL)

👉 If IGNORE folder was already tracked before .gitignore, remove it:

git rm -r --cached IGNORE

git commit -m "Removed IGNORE folder from tracking"

git push origin GIT-EXAM-1